PulseWallet — What We Have Done So Far

Project: PulseWallet
Purpose: A personal financial planning and transaction fraud detection system.

1. Backend Foundation

We created the Spring Boot backend with:

- Java + Spring Boot
- PostgreSQL database
- Maven
- Environment/profile configuration
- REST API structure
- Git/GitHub version control
- Database migrations using Flyway

2. User Authentication & Transactions

We implemented:

- User registration and login
- JWT-based authentication
- Password encryption using BCrypt
- User-specific data protection
- Transaction management
- Income and expense tracking
- Categories
- Budgets

So each user can securely manage their own financial data.

3. Financial Planning

We added a financial planning engine that analyzes:

- Income
- Expenses
- Monthly surplus

- Spending categories
- Budgets
- Historical spending

Based on this, PulseWallet generates:

- Spending forecasts
- Budget recommendations
- Savings recommendations
- Investment suggestions
- Financial advice

4. Fraud Detection

We built a separate **Python FastAPI fraud detection service** using machine learning.

We experimented with models including Random Forest and XGBoost and selected **XGBoost** as the production model.

The system:

Transaction

↓

Fraud Detection Model

↓

Fraud Probability

↓

Risk Score / Risk Level

↓

Alert

The Spring Boot backend communicates with the FastAPI fraud service.

5. Fraud Alerts

We added persistent fraud alerts so suspicious transactions can be stored and displayed to the user.

We also added configurable fraud-score threshold handling.

6. System Integration

When a transaction is created, it can go through the fraud detection system and generate an alert when the risk is high.

We also added dashboard-summary and notification functionality.

7. Frontend Dashboard

We built the frontend with:

- Login/signup
- Dashboard
- Transactions
- Budgets
- Alerts
- Financial recommendations
- Charts
- Loading/error/empty states
- Currency handling

8. Financial Goals

We added goal-based planning.

Users can create goals such as:

- Bike
- Car
- House
- Trip
- Education
- Emergency fund
- Custom goals

PulseWallet calculates:

- Goal progress
- Remaining amount
- Required monthly saving

- Remaining months
- Goal status

9. Savings & Investment Planning

We added a **Financial Plan** that uses actual income, expenses and active goals to calculate a recommended allocation between:

- Expenses
- Savings
- Investments
- Emergency/buffer
- Flexible spending

We also added an investment profile for risk/horizon preferences.

10. Credit Card Intelligence

We added credit-card management and intelligence.

The system can track safe card metadata such as:

- Card issuer
- Card name
- Last 4 digits
- Credit limit
- Outstanding
- Statement date
- Due date
- Rewards/benefits

It also calculates/considers:

- Credit utilization
- Repayment capacity
- Spending risk

11. Purchase Advisor

We added the foundation for a **Purchase Advisor**.

The idea is:

User wants to buy something

↓

Enter amount + category

↓

Check financial situation

↓

Compare eligible cards

↓

Consider rewards/benefits

↓

Check credit impact

↓

Give recommendation

12. Linked Payment Accounts

Most recently, we added **Linked Payment Accounts**.

Users can manage connected account records such as:

- Bank
- UPI
- Payment app

with:

- Provider
- Account name
- Account type
- Masked identifier
- Connection status
- Connected date
- Last sync date

This is currently a **connection-management foundation**, not direct PhonePe/Google Pay/bank login integration.

13. Testing

We have been testing both backend and frontend after major changes.

The latest major backend verification reached:

149 tests — 0 failures — 0 errors

Frontend build also passed successfully.

🎤 **Very Short Version for Your Guide**

You can say:

"Our project PulseWallet is a personal financial planning and transaction fraud detection system. We developed the backend using Java Spring Boot and PostgreSQL with JWT authentication and transaction management. We implemented a financial planning engine for budgeting, spending forecasting, savings, investment planning and goal-based recommendations. We also developed a separate FastAPI machine-learning service using XGBoost for transaction fraud detection and integrated it with the Spring Boot backend to generate fraud alerts. On top of this, we added financial goals, credit-card intelligence, purchase recommendations and linked payment-account management. The frontend provides the dashboard and interfaces for all these modules. We have also performed backend and frontend testing after the implementations."

Current status: the major functional modules up to **Linked Payment Accounts** have been implemented and verified; the frontend is now the part we are planning to redesign/restructure.

what stack and how have been we used it

Yes. For your guide, explain the stack in terms of **what technology we used + why + where we used it**.

PulseWallet Tech Stack

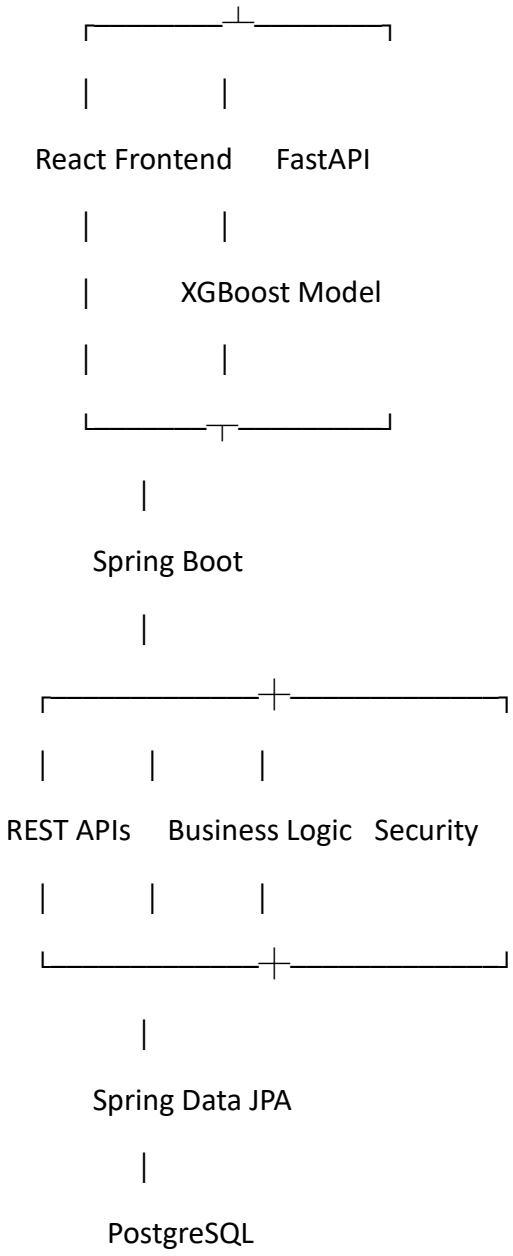
Layer	Technology	How we used it
Backend	**Java**	Main programming language for the backend
Backend Framework	**Spring Boot 4.1**	Built REST APIs and backend business logic

Layer	Technology	How we used it
Security	**Spring Security + JWT**	Login, authentication and protected APIs
Database	**PostgreSQL 17**	Stores users, transactions, budgets, goals, cards, alerts, etc.
ORM	**Spring Data JPA / Hibernate**	Connects Java entities with PostgreSQL tables
Database Migration	**Flyway**	Manages database schema changes through migrations V1–V13
Build Tool	**Maven**	Dependency management, compilation and testing
Fraud ML	**Python**	Developed the fraud-detection model/service
ML Framework	**XGBoost + scikit-learn**	Trained and evaluated the fraud detection model
Data Processing	**Pandas / NumPy**	Dataset preprocessing and analysis
Fraud API	**FastAPI**	Exposes the trained ML model through /fraud-check
Frontend	**React + JavaScript/JSX** *(current implementation)*	Dashboard and user interface
Frontend Build	**Vite**	Runs/builds the current React frontend
Charts	**Chart.js** / frontend charting	Financial/spending visualization where used
Version Control	**Git + GitHub**	Source-code management and collaboration
API Communication	**REST + JSON**	Frontend ↔ Spring Boot ↔ FastAPI communication

How everything connects

PULSEWALLET

|



How we use each major technology

1. Java + Spring Boot

Spring Boot is the **main backend**. We use it for:

- Authentication
- Transactions
- Goals
- Budgets
- Financial planning
- Recommendations

- Credit-card intelligence
- Fraud integration
- Alerts
- Linked accounts

For example:

Frontend

↓

GET /api/goals

↓

GoalController

↓

GoalService

↓

GoalRepository

↓

PostgreSQL

---

2. PostgreSQL

This is our **persistent database**.

We store application data such as:

User

Transaction

Category

Budget

Financial Goal

Investment Profile

Credit Card

Fraud Alert

Notification

Linked Payment Account

We don't directly write SQL for every operation. JPA/Hibernate handles much of the database interaction.

---

3. Spring Data JPA + Hibernate

We create Java entities such as:

@Entity

public class FinancialGoal {

...

}

Hibernate maps that Java object to a PostgreSQL table.

So conceptually:

Java Entity

↓

Hibernate/JPA

↓

PostgreSQL Table

---

4. Flyway

Whenever we introduce a new database feature, we create a migration.

For example:

V10 → Financial Goals

V11 → Investment Profiles

V12 → Credit Cards

V13 → Linked Payment Accounts

This makes database changes controlled and reproducible.

---

5. Python + XGBoost

Python is used specifically for the **fraud detection/ML side**.

We used the credit-card fraud dataset, performed preprocessing and experimented with models.

XGBoost was selected as the production model.

Transaction Features

↓

Preprocessing

↓

XGBoost

↓

Fraud Probability

↓

Risk Score

↓

Spring Boot

↓

Fraud Alert

---

6. FastAPI

FastAPI acts as the bridge between our Java application and the Python ML model.

Spring Boot sends transaction information to:

POST /fraud-check

FastAPI processes it using the trained XGBoost model and returns the fraud result.

---

7. React + JavaScript

Important: this is our **current frontend stack**.

We used React because it helps us build dynamic pages such as:

- Dashboard
- Transactions
- Goals
- Financial Plan
- Credit Cards
- Alerts
- Linked Accounts

However, as you just said, **you don't know React and want to rebuild the frontend using plain HTML + CSS + JavaScript**.

So after the rebuild, the frontend stack will become:

HTML

CSS

Vanilla JavaScript

Fetch API

while the **Spring Boot + PostgreSQL + FastAPI + XGBoost backend remains the same**.

🎤 **Short answer for your guide**

"Our PulseWallet stack consists mainly of Java Spring Boot, PostgreSQL, Python FastAPI and XGBoost, with a React-based frontend currently. Spring Boot is used to develop our REST APIs and financial business logic, PostgreSQL stores the application data, JPA and Hibernate provide database connectivity, and Flyway manages database migrations. For fraud detection, we use Python with XGBoost and expose the trained model through FastAPI. The frontend communicates with Spring Boot through REST APIs and JSON. We use JWT with Spring Security for authentication and GitHub for version control."

One thing to remember for your guide: **don't say that XGBoost itself handles the whole fraud system**. XGBoost is the **ML model**, FastAPI exposes it, and Spring Boot integrates its result into PulseWallet.